Trabajo Práctico Integrador II

Asignatura: Arquitectura Web – FP-UNA 2026

De monolito a Cloud Native: descomposición, contenerización, despliegue distribuido, resiliencia y observabilidad de BookStack

Docente: José Rodrigo Benítez Paredes, Msc
Institución: Facultad Politécnica – Universidad Nacional de Asunción (FP-UNA)
Continuidad de: TP Integrador I – Fases 1 a 4 (Despliegue y Aseguramiento de BookStack)
Puntaje total: 100 puntos en tres bloques · Fecha de Entrega: Segundo Parcial - 2026

Resumen

El presente documento plantea las **Fases 5, 6 y 7** del proyecto integrador de la asignatura **Arquitectura Web**, como continuidad natural del TP Integrador I. Sobre la misma infraestructura ya desplegada, asegurada con TLS 1.3 y auditada, la plataforma **BookStack** evoluciona de una aplicación monolítica a una **aplicación nativa de la nube (CNA)** mediante el patrón *Strangler Fig*. Se extraen cuatro funcionalidades —búsqueda, auditoría, exportación y gestión de medios— a microservicios contenerizados que interactúan con el monolito a través de su API REST y sus Webhooks, sin modificar su código fuente. El trabajo se evalúa sobre 100 puntos distribuidos en tres bloques: **(I) Diseño y descomposición** (25 pts), **(II) Construcción, contenerización y despliegue distribuido** en dos o tres nodos (45 pts) y **(III) Operación cloud native**, que abarca experimentos de caos, escalabilidad comparada contra la línea base de la Fase 4, observabilidad con métricas, logs y trazas, seguridad de la cadena de suministro y defensa oral (30 pts).

Índice

1. Contexto y Continuidad con el TP Integrador I 3
1.1. Punto de Partida: Línea Base Heredada del TP I 3
2. Objetivos, Alcance y Criterios de Aceptación 3
3. Arquitectura Objetivo (“To Be”) 4
3.1. Funcionalidades a Extraer del Monolito 5
3.2. Componentes de Plataforma 6
3.3. Topología de Despliegue 6
3.4. Interacciones Obligatorias 6
4. Bloque I – Fase 5: Diseño y Descomposición del Monolito (25 pts) 8
4.1. A1 – Análisis del Monolito y Mapa de Contextos (6 pts) 8
4.2. A2 – Registro de Decisiones de Arquitectura ADR-006 a ADR-010 (5 pts) 8
4.3. A3 – Contratos de API y de Eventos (5 pts) 9

4.4. A4 – Modelo C4 y Diagramas de Secuencia (5 pts) 9
4.5. A5 – Plan de Migración y Matriz de Riesgos (4 pts) 9
4.6. Rúbrica del Bloque I 10
5. Bloque II – Fase 6: Construcción, Contenerización y Despliegue Distribuido (45 pts) 11
5.1. B1 – Infraestructura Multi-Nodo (6 pts) 11
5.2. B2 – Microservicios (20 pts) 11
5.3. B3 – Contenerización con Buenas Prácticas (6 pts) 12
5.4. B4 – Integración con el Monolito y el Broker (7 pts) 12
5.5. B5 – API Gateway y Orquestación (6 pts) 12
5.6. Rúbrica del Bloque II 13
6. Bloque III – Fase 7: Operación Cloud Native, Resiliencia, Seguridad y Defensa (30 pts) 14
6.1. C1 – Resiliencia y Experimentos de Caos (8 pts) 14
6.2. C2 – Escalabilidad y Rendimiento Comparado (6 pts) 14
6.3. C3 – Observabilidad: Métricas, Logs y Trazas (7 pts) 14
6.4. C4 – Seguridad de la Plataforma y Cadena de Suministro (4 pts) 15
6.5. C5 – Matriz de Pruebas, Demostración y Defensa (5 pts) 15
6.6. Rúbrica del Bloque III 15
7. Distribución de Puntaje, Entregables y Reglas 15
7.1. Distribución de los 100 Puntos 15
7.2. Entregables 16
7.3. Hitos de Entrega 16
7.4. Reglas 16
8. Matriz de Evidencias Esperadas 16

1. Contexto y Continuidad con el TP Integrador I

Este trabajo continúa el Proyecto Integrador I (Fases 1 a 4) y agrega las **Fases 5, 6 y 7**: el mismo BookStack monolítico, ya desplegado, asegurado con TLS 1.3 y auditado, se evoluciona hacia una **aplicación nativa de la nube (CNA)** sin apagar el monolito.

La estrategia obligatoria es el patrón **Strangler Fig**: el monolito sigue atendiendo la gestión documental, y alrededor de él se construyen microservicios que le van "estrangulando" funcionalidades. La integración se hace por las vías que BookStack ya ofrece —su **API REST** con tokens y sus **Webhooks** de eventos— sin modificar el código fuente de BookStack.

1.1. Punto de Partida: Línea Base Heredada del TP I

Elemento	Estado al cierre de la Fase 4
VM	Ubuntu Server 26.04 LTS, 2 vCPU, 4 GB RAM, 20 GB, modo puente, `192.168.100.23`
Edge	Nginx 1.28 con TLS 1.3, HSTS, CSP, redirección 301, `bookstack.local`
Aplicación	BookStack v26.05.4 sobre PHP 8.5-FPM (socket Unix)
Datos	MariaDB 11.8 en `127.0.0.1:3306`
Seguridad	UFW con 22/80/443, `server_tokens off`, cookies Secure/HttpOnly
Rendimiento	~50 req/s con `ab -n 1000 -c 10`, 0 errores; 23,7 ms promedio de TLS + respuesta

Tabla 1: Línea base de infraestructura y rendimiento heredada del TP Integrador I.

Esa línea base es la referencia contra la que se compararán todas las mediciones nuevas. Se trabaja en el **mismo repositorio** (`tp-arqweb-equipo03`), en una carpeta nueva `cna/`, y el informe continúa la numeración de secciones, figuras (F5-E1…), ADR (ADR-006…) y la matriz de pruebas (P26…).

2. Objetivos, Alcance y Criterios de Aceptación

Objetivo general: extraer al menos **tres funcionalidades** del monolito BookStack a microservicios contenerizados, desplegados en dos o tres nodos, que interactúen entre sí y con el monolito de forma síncrona (REST) y asíncrona (eventos), cumpliendo los principios de la metodología *Twelve-Factor App*.

Objetivos específicos:

1. Analizar el monolito e identificar contextos acotados (*bounded contexts*) candidatos a extracción, justificando cada decisión con ADR.
2. Construir cada microservicio como contenedor inmutable, sin estado local, configurable por variables de entorno y con su propio almacén de datos (*database per service*).
3. Desplegar la solución en una topología distribuida de 2 o 3 VM (o, como mínimo, 1 VM con red interna de contenedores), detrás del Nginx existente como **API Gateway**.
4. Implementar comunicación asíncrona con un broker de mensajes y garantizar la entrega confiable de eventos.
5. Demostrar las propiedades cloud native: escalado horizontal, autorrecuperación, degradación elegante, observabilidad (métricas, logs y trazas) y seguridad en la cadena de suministro.

Criterios de aceptación mínimos. Sin ellos el trabajo no se aprueba (ver sección 7.1):

ID	Criterio	Umbral
CA-1	Microservicios extraídos y funcionando	≥ 3 servicios, cada uno con su propio contenedor y datos
CA-2	El monolito sigue operativo	Login, crear libro y editar página funcionan igual que en la Fase 4
CA-3	Interacción asíncrona extremo a extremo	Editar una página en BookStack actualiza el índice de búsqueda en < 5 s
CA-4	Gateway único con TLS 1.3	Todo el tráfico externo entra por 443; ningún puerto de servicio expuesto al exterior
CA-5	Degradación elegante	Con cualquier microservicio caído, el monolito sigue respondiendo 200 OK
CA-6	Cero pérdida de eventos	Reiniciar un consumidor no pierde eventos publicados durante su caída
CA-7	Reproducibilidad	`docker compose up -d` (o equivalente) levanta todo desde el repositorio limpio

Tabla 2: Criterios de aceptación globales del TP Integrador II.

3. Arquitectura Objetivo (“To Be”)

La arquitectura objetivo reparte la solución en tres nodos: el monolito queda en su VM original, los microservicios y el broker en un segundo nodo, y la observabilidad en un tercero. Este último es opcional si la RAM del anfitrión no alcanza; en ese caso, los nodos 2 y 3 se fusionan. La Figura 1 actualiza el diagrama “As Built” de la Figura 1 del TP I.

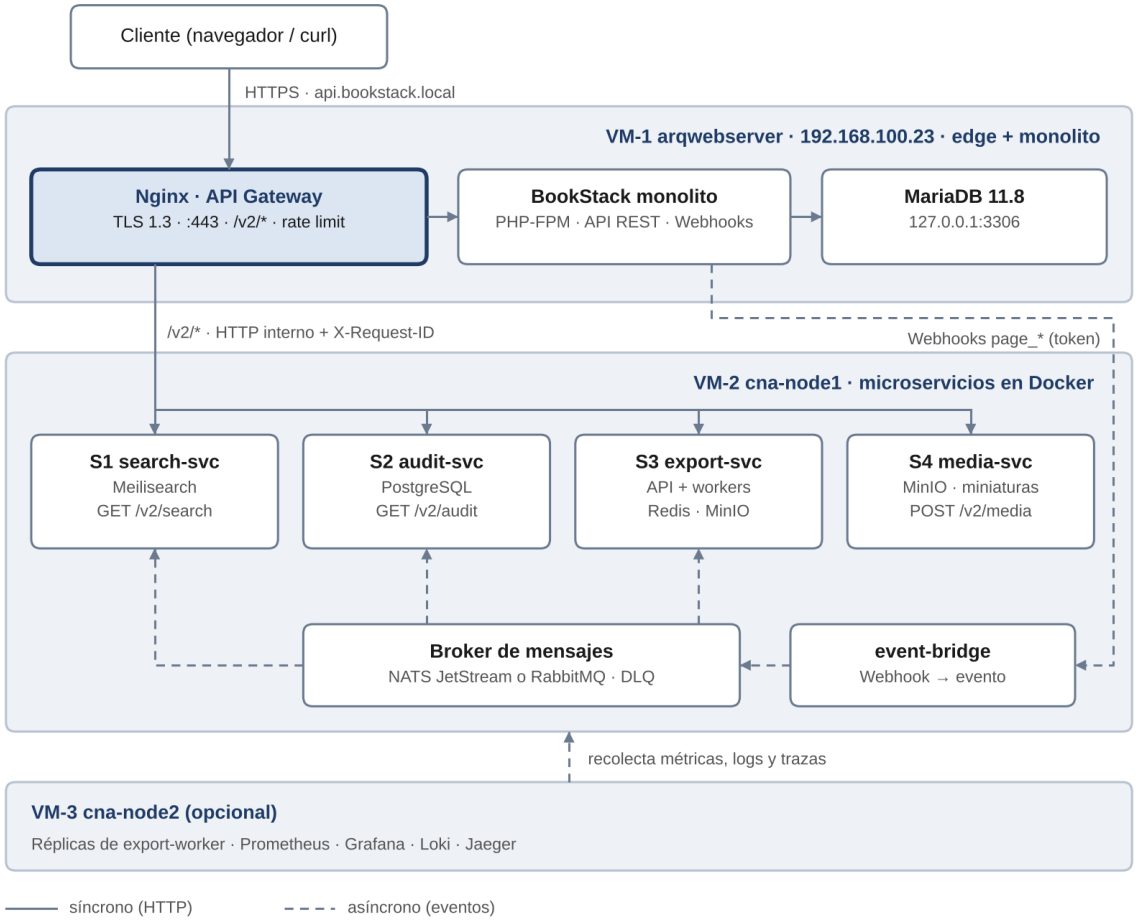


Figura 1: Arquitectura objetivo de tres nodos con gateway único, cuatro microservicios y comunicación síncrona y asíncrona.

El monolito no conoce a los microservicios: solo emite Webhooks hacia `event-bridge`, y los servicios S1 y S3 le leen datos por su API REST con un token propio. MariaDB nunca sale de `127.0.0.1`.

3.1. Funcionalidades a Extraer del Monolito

Los servicios **S1, S2 y S3 son obligatorios**. El **S4 es obligatorio para optar a la nota máxima** del criterio B2 (ver Bloque II); el equipo puede proponer otro S4 equivalente con aprobación previa del docente.

Servicio	Funcionalidad que “estrangula”	Interacción	Almacén propio	Endpoints mínimos
S1 `search-svc`	Búsqueda de texto completo sobre libros y páginas	Consume eventos `page.*` + API REST síncrona	Meilisearch o PostgreSQL FTS	`GET /v2/search?q=`
S2 `audit-svc`	Registro de actividad y auditoría (hoy tabla `activities`) + notificaciones	100 % asíncrono (consumidor)	PostgreSQL o MongoDB	`GET /v2/audit?entity=&id=`
S3 `export-svc`	Exportación de páginas y libros a PDF o Markdown	Asíncrono por trabajos: API + cola + *workers*	Redis (jobs) + MinIO (archivos)	`POST /v2/exports` → 202, `GET /v2/exports/{id}`

Servicio	Funcionalidad que "estrangula"	Interacción	Almacén propio	Endpoints mínimos
S4 `media-svc`	Imágenes y adjuntos en almacenamiento de objetos con miniaturas	REST síncrono + evento `media.uploaded`	MinIO (compatible S3)	`POST /v2/media`, `GET /v2/media/{id}`

Tabla 3: Funcionalidades del monolito a extraer como microservicios. Todos exponen además `/healthz`, `/readyz` y `/metrics`.

3.2. Componentes de Plataforma

Los siguientes componentes son necesarios para la interacción, pero **no cuentan** como servicio extraído:

- **API Gateway:** el Nginx existente, ahora con un *server block* `api.bookstack.local` que enruta `/v2/*` a los servicios por la red interna.
- `event-bridge`**:** servicio mínimo que recibe los Webhooks de BookStack, les asigna un `event_id` y los publica en el broker. Desacopla el monolito del broker.
- **Broker de mensajes:** NATS JetStream o RabbitMQ, con colas o *streams* durables.
- **Observabilidad:** Prometheus + Grafana (métricas), Loki o equivalente (logs), Jaeger u OpenTelemetry Collector (trazas).

El lenguaje es libre por servicio (Python/FastAPI, Node.js, Go, Java/Quarkus). Se valora que al menos dos servicios usen lenguajes distintos, demostrando la independencia tecnológica que habilita esta arquitectura.

3.3. Topología de Despliegue

Nodo	Hostname / IP	Rol	Contenido
VM-1 (existente)	`arqwebserver` · 192.168.100.23	Edge + monolito	Nginx (gateway TLS), PHP-FPM + BookStack, MariaDB
VM-2 (clon enlazado)	`cna-node1` · DHCP o estática	Servicios	Docker Engine; S1–S4, `event-bridge`, broker y almacenes de cada servicio
VM-3 (opcional)	`cna-node2` · DHCP o estática	Réplicas + observabilidad	Réplicas de `export-worker`, Prometheus, Grafana, Loki, Jaeger

Tabla 4: Topología de despliegue sugerida.

Orquestación aceptada, de menor a mayor complejidad: **Docker Compose** en un nodo (base), **Docker Swarm** en 2–3 nodos (recomendado) o **k3s** (suma puntos en B5). Con 4 GB de RAM por VM, cada contenedor debe declarar límites de memoria.

3.4. Interacciones Obligatorias

1. **Edición → indexación (asíncrona):** el usuario edita una página → BookStack dispara el Webhook `page_update` → `event-bridge` publica `bookstack.page.updated` → S1 consume, consulta `GET /api/pages/{id}` de BookStack con su token y reindexa → S2 consume el mismo evento y registra la auditoría.
2. **Búsqueda (síncrona):** cliente → `https://api.bookstack.local/v2/search?q=nginx` → gateway → S1 → resultados JSON con enlaces a las páginas del monolito.
3. **Exportación (asíncrona por trabajos):** `POST /v2/exports {page_id, format}` → S3 responde `202 Accepted` + `job_id` → un *worker* toma el trabajo, pide la exportación a la API

de BookStack, guarda el archivo en MinIO y marca el job como `done` → `GET /v2/exports/{job_id}` devuelve el estado y una URL prefirmada.

4. **Correlación:** toda petición lleva la cabecera `X-Request-ID` generada en el gateway y propagada por todos los servicios y mensajes, de modo que una edición puede seguirse de punta a punta en logs y trazas.

4. Bloque I – Fase 5: Diseño y Descomposición del Monolito (25 pts)

El Bloque I se aprueba cuando el equipo puede justificar, antes de escribir código, **qué** se extrae, **por qué**, **cómo se comunican** las partes y **qué se rompe** si algo falla. Ningún contenedor se construye en esta fase: el producto es diseño verificable.

4.1. A1 – Análisis del Monolito y Mapa de Contextos (6 pts)

- Inventariar los módulos funcionales de BookStack (contenido, búsqueda, actividad, exportación, imágenes/adjuntos, usuarios/permisos) y mapear qué tablas de MariaDB usa cada uno (`SHOW TABLES`, consultas a `information_schema`).
- Construir una **matriz de acoplamiento** módulo × tabla y marcar las tablas compartidas, que son las que impiden una extracción directa.
- Dibujar el **mapa de contextos** (DDD) con las relaciones entre contextos (cliente/proveedor, capa anticorrupción, etc.).
- Evidencias: **F5-E1** consulta a `information_schema` con las tablas por módulo; **F5-E2** matriz de acoplamiento; **F5-E3** mapa de contextos.

4.2. A2 – Registro de Decisiones de Arquitectura ADR-006 a ADR-010 (5 pts)

Cada ADR se redacta con el mismo formato que los ADR-001 a ADR-005 del TP I (Estatus, Contexto, Decisión, Consecuencias) e incluye **al menos una alternativa descartada**. Los recuadros siguientes plantean la decisión que cada equipo debe tomar y justificar:

ADR-006: Estrategia de Migración e Integración con el Monolito

Contexto: el monolito BookStack debe seguir operando mientras se extraen funcionalidades.

Decisión a justificar: patrón Strangler Fig con integración exclusiva por API REST + Webhooks, sin tocar el código de BookStack.

ADR-007: Selección del Broker de Mensajes

Contexto: se requiere entrega confiable de eventos entre `event-bridge` y los consumidores.

Decisión a justificar: NATS JetStream vs. RabbitMQ vs. Redis Streams, considerando durabilidad, consumo de RAM y operación.

ADR-008: Plataforma de Orquestación

Contexto: la infraestructura disponible son VM de 2 vCPU y 4 GB de RAM sobre VMware Workstation.

Decisión a justificar: Docker Compose vs. Docker Swarm vs. k3s según los recursos reales medidos.

ADR-009: Base de Datos por Servicio

Contexto: los microservicios necesitan persistencia propia sin acoplarse al esquema de BookStack.

Decisión a justificar: *database per service* y prohibición de acceso directo de los microservicios a MariaDB.

ADR-010: Consistencia Eventual e Idempotencia

Contexto: los datos del índice y de la auditoría se actualizan de forma asíncrona.

Decisión a justificar: modelo de consistencia eventual, idempotencia por `event_id` y manejo de mensajes fallidos (DLQ).

4.3. A3 – Contratos de API y de Eventos (5 pts)

- Un archivo **OpenAPI 3.1** por servicio (`cna/contracts/<servicio>.openapi.yaml`) con esquemas, códigos de respuesta (200, 202, 400, 404, 503) y ejemplos.
- Un archivo **AsyncAPI 3.0** con los canales de eventos (`bookstack.page.created`, `.updated`, `.deleted`, `export.requested`, `export.completed`, `media.uploaded`).
- Un **sobre de evento** común con `event_id` (UUID), `type`, `version`, `occurred_at` (ISO 8601), `request_id` y `payload`.
- Política de **versionado**: rutas `/v2/` y campo `version` en los eventos; explicar cómo se agregaría un `v3` sin romper consumidores.
- Evidencia: **F5-E4** contratos validados con un linter (Spectral, `swagger-cli` o `asyncapi validate`) sin errores.

4.4. A4 – Modelo C4 y Diagramas de Secuencia (5 pts)

- **C4 nivel 1 (contexto) y nivel 2 (contenedores)** de la solución completa, mostrando nodos, puertos y protocolos (actualiza la Figura 1 del TP I).
- **Tres diagramas de secuencia**, uno por interacción obligatoria (sección 3.4), incluyendo el camino de error de cada uno.
- Herramientas sugeridas: Structurizr, PlantUML, Mermaid o draw.io, con el fuente versionado en `cna/docs/`.
- Evidencias: **F5-E5** C4 contexto; **F5-E6** C4 contenedores; **F5-E7** diagramas de secuencia.

4.5. A5 – Plan de Migración y Matriz de Riesgos (4 pts)

- Orden de extracción justificado (de menor a mayor riesgo) y criterio de “hecho” de cada paso.
- Estrategia de **reversión**: cómo desactivar una ruta `/v2/*` en el gateway y volver al monolito en menos de 1 minuto.
- Matriz de riesgos (probabilidad × impacto) con al menos: consistencia eventual visible al usuario, consumo de RAM, latencia de red entre VM, secretos compartidos y punto único de falla en el gateway.

4.6. Rúbrica del Bloque I

Ítem	Pts	Logro completo	Logro parcial (≈ 50 %)
A1 Análisis y contextos	6	Tablas por módulo verificadas en la BD real; acoplamientos explicados	Mapa sin respaldo en la BD o sin tablas compartidas identificadas
A2 ADR-006 a 010	5	Cinco ADR con alternativa descartada y consecuencias negativas reconocidas	ADR sin alternativas o solo con consecuencias positivas
A3 Contratos	5	OpenAPI + AsyncAPI validados, sobre de evento y versionado	Contratos incompletos o sin validar
A4 C4 y secuencias	5	C4 L1/L2 coherentes con lo desplegado; secuencias con caminos de error	Diagramas genéricos o sin caminos de error
A5 Migración y riesgos	4	Orden, reversión planificada y riesgos con mitigación	Lista de riesgos sin mitigación ni orden
Total Bloque I	**25**		

Tabla 5: Rúbrica de evaluación del Bloque I – Fase 5.

5. Bloque II – Fase 6: Construcción, Contenerización y Despliegue Distribuido (45 pts)

El Bloque II es el de mayor peso porque es donde el diseño se vuelve sistema: al cerrarlo, las cuatro interacciones de la sección 3.4 deben funcionar en vivo sobre la infraestructura multi-nodo, con el monolito intacto.

5.1. B1 – Infraestructura Multi-Nodo (6 pts)

- Crear VM-2 (y VM-3 si aplica) como **clon enlazado** de la VM base, con hostname e IP propios; documentar los recursos asignados.
- Instalar Docker Engine y el plugin Compose; si se usa Swarm o k3s, formar el clúster y mostrar los nodos activos.
- Extender la política UFW de la Fase 4: en VM-2/VM-3 solo se aceptan conexiones **desde 192.168.100.23** a los puertos de servicio; el exterior sigue entrando únicamente por 443 de VM-1.
- Agregar `api.bookstack.local` a `/etc/hosts` del anfitrión y **reemitir el certificado X.509** con ambos nombres en el SAN.
- Evidencias: **F6-E1** `docker info` / `docker node ls` / `kubectl get nodes`; **F6-E2** `ufw status verbose` de cada nodo; **F6-E3** inspección del nuevo certificado con dos entradas DNS en el SAN.

5.2. B2 – Microservicios (20 pts)

Requisitos comunes a **todos** los servicios, que se verifican en cada uno:

- Configuración solo por variables de entorno; ningún secreto en el código ni en la imagen.
- Sin estado local: el contenedor puede destruirse y recrearse sin perder datos.
- `GET /healthz` (vivo) y `GET /readyz` (listo: sus dependencias responden), separados.
- `GET /metrics` en formato Prometheus con, como mínimo, un contador de peticiones por código y un histograma de latencia.
- Logs en JSON a `stdout` con `timestamp`, `level`, `service`, `request_id` y `message`.
- Propagación de `X-Request-ID` en HTTP y del campo `request_id` en los eventos.
- Al menos 3 pruebas automatizadas por servicio (unitarias o de contrato), ejecutables con un solo comando.

Sub-ítem	Pts	Requisitos específicos	Evidencia
B2.1 S1 `search-svc`	6	Indexación inicial completa (recorre `GET /api/pages` paginado); reindexación por evento; búsqueda con resaltado y enlace a la página del monolito; elimina del índice ante `page.deleted`	**F6-E4** búsqueda vía gateway antes y después de editar una página
B2.2 S2 `audit-svc`	5	Consumidor durable; registra quién, qué, cuándo y `request_id`; consulta filtrable por entidad y rango de fechas; idempotente ante eventos duplicados	**F6-E5** mismo evento enviado dos veces → un solo registro
B2.3 S3 `export-svc`	6	API devuelve 202 + `job_id`; *worker* separado del API; estados `queued`, `running`, `done`, `failed`; archivo en MinIO con URL prefirmada que expira	**F6-E6** ciclo completo de un job con `curl`; **F6-E7** objeto en la consola de MinIO

Sub-ítem	Pts	Requisitos específicos	Evidencia
B2.4 S4 `media-svc`	3	Subida de imagen, miniatura generada y evento `media.uploaded` publicado	**F6-E8** subida y descarga de miniatura vía gateway
Total B2	**20**		

Tabla 6: Requisitos específicos y evidencias por microservicio.

5.3. B3 – Contenerización con Buenas Prácticas (6 pts)

- Dockerfile **multi-stage** por servicio, imagen base mínima (alpine, distroless o slim) y **usuario no root**.
- Instrucción `HEALTHCHECK` o *probes* equivalentes del orquestador.
- Etiquetas con versión semántica (`search-svc:1.0.0`) más el hash corto del commit; prohibido desplegar `:latest`.
- Límites de CPU y memoria declarados para cada contenedor, sumando menos que la RAM del nodo.
- Sistema de archivos de solo lectura (`read_only: true`) donde el servicio lo permita.
- Evidencias: **F6-E9** tabla con nombre, etiqueta y tamaño de cada imagen (`docker images`); **F6-E10** `docker inspect` mostrando `User`, límites y healthcheck.

5.4. B4 – Integración con el Monolito y el Broker (7 pts)

- Crear en BookStack un **usuario de servicio** con un rol que solo tenga el permiso de acceso a la API y lectura de contenido; generar su token y entregarlo a los servicios como secreto.
- Configurar en BookStack los **Webhooks** de creación, actualización y eliminación de páginas apuntando a `event-bridge` por la red interna, con un token compartido en la URL que `event-bridge` valida (rechazo con 401 si no coincide).
- `event-bridge` convierte el Webhook al sobre de evento definido en A3 y lo publica en el broker.
- *Streams* o colas **durables** con confirmación manual (*ack*) y cola de mensajes fallidos (**DLQ**) tras N reintentos.
- Ningún microservicio se conecta a MariaDB: se verifica con `ss -tnp` en VM-1 durante la carga.
- Evidencias: **F6-E11** configuración del Webhook en BookStack; **F6-E12** evento recibido en el broker (CLI de NATS o consola de RabbitMQ); **F6-E13** mensaje forzado a la DLQ.

5.5. B5 – API Gateway y Orquestación (6 pts)

- Nuevo *server block* en el Nginx de VM-1 para `api.bookstack.local` con TLS 1.3, HSTS y las mismas cabeceras de seguridad de la Fase 4.
- `upstream` por servicio, `proxy_connect_timeout` y `proxy_read_timeout` explícitos, generación de `X-Request-ID` (`$request_id`) y **rate limiting** (`limit_req`) sobre `/v2/`.
- Respuesta JSON uniforme `503` cuando un servicio no está disponible, en lugar de la página HTML de Nginx.
- Despliegue completo con **un comando** (`docker compose up -d`, `docker stack deploy` o `kubectl apply -k`) desde el repositorio limpio; secretos mediante Docker secrets, Kubernetes Secrets o `.env` no versionado (con `.env.example` sí versionado).

- Uso de Swarm o k3s con al menos dos nodos: hasta +2 pts dentro de este ítem frente a Compose en un nodo (máximo 6).
- Evidencias: **F6-E14** `nginx -t` y extracto del *server block*; **F6-E15** respuesta 429 al superar el límite; **F6-E16** despliegue completo en un comando con todos los servicios `healthy`.

5.6. Rúbrica del Bloque II

Ítem	Pts	Logro completo	Logro parcial (≈ 50 %)
B1 Infraestructura multi-nodo	6	Nodos operativos, UFW restringido por origen, certificado con doble SAN	Un solo nodo sin justificación o puertos de servicio abiertos al exterior
B2 Microservicios	20	Cada servicio cumple los requisitos comunes y específicos	Servicios funcionando sin healthz, metrics o logs estructurados
B3 Contenerización	6	Multi-stage, no root, versionado, límites y healthcheck en todas las imágenes	Imágenes funcionales con root o `:latest`
B4 Integración y broker	7	Webhooks autenticados, colas durables, DLQ y cero acceso a MariaDB	Integración por *polling* o sin durabilidad
B5 Gateway y orquestación	6	Gateway con TLS, rate limit, 503 JSON y despliegue en un comando	Despliegue manual por pasos o gateway sin límites
Total Bloque II	**45**		

Tabla 7: Rúbrica de evaluación del Bloque II – Fase 6.

6. Bloque III – Fase 7: Operación Cloud Native, Resiliencia, Seguridad y Defensa (30 pts)

El Bloque III demuestra que el sistema no solo funciona, sino que **sobrevive a fallos, escala y se puede observar**, con números comparados contra la línea base de la Fase 4. Es la continuación directa de las secciones 7.3 y 7.4 del TP I, ahora sobre una arquitectura distribuida.

6.1. C1 – Resiliencia y Experimentos de Caos (8 pts)

Cada experimento se documenta con hipótesis, procedimiento, resultado esperado, resultado obtenido y tiempo de recuperación medido:

ID	Experimento	Resultado esperado
X1	`docker kill` del contenedor de S1	Monolito responde 200; `/v2/search` responde 503 JSON; el orquestador lo recrea solo; se mide el MTTR
X2	Detener S2 durante 2 minutos mientras se editan 20 páginas	Al volver, S2 procesa los 20 eventos pendientes: cero pérdida (CA-6)
X3	Matar un *worker* de S3 a mitad de un job	El job vuelve a la cola y otro *worker* lo completa; no queda en `running` indefinidamente
X4	Apagar VM-2 completa	BookStack sigue operativo (CA-5); al encender VM-2 el sistema converge sin intervención manual
X5	Latencia artificial de 2 s hacia la API de BookStack (`tc netem`)	S1 aplica timeout, reintentos con *backoff* exponencial e interruptor de circuito; no satura sus hilos

Tabla 8: Experimentos de caos obligatorios.

- Evidencias: **F7-E1** a **F7-E5**, una por experimento, con marcas de tiempo visibles.

6.2. C2 – Escalabilidad y Rendimiento Comparado (6 pts)

- Medir con **k6** (preferido) o `ab` la búsqueda del monolito (`/search?term=`) frente a `GET /v2/search` de S1, ambas por HTTPS a través del gateway, con la misma concurrencia de la Fase 4 (c=10) y una mayor (c=50).
- Escalar `export-worker` de 1 a 3 réplicas y medir trabajos completados por minuto con una ráfaga de 50 exportaciones.
- Registrar el consumo con `docker stats` y `htop` en cada nodo durante las pruebas.
- Presentar una tabla comparativa contra la línea base (~50 req/s, 0 errores) e interpretar los resultados: el costo de los saltos de red adicionales también es un hallazgo válido.
- Evidencias: **F7-E6** resultados de k6; **F7-E7** escalado de *workers*; **F7-E8** consumo por nodo.

6.3. C3 – Observabilidad: Métricas, Logs y Trazas (7 pts)

- Tablero de Grafana con las **métricas RED** (tasa, errores, duración p50/p95/p99) por servicio y la profundidad de cada cola del broker.
- Agregación de logs: filtrar por un `request_id` y ver la misma operación en gateway, `event-bridge`, S1 y S2.
- **Traza distribuida** de la interacción edición → indexación en Jaeger (instrumentación OpenTelemetry), mostrando cada tramo y su duración.
- Al menos **una regla de alerta** (por ejemplo, mensajes en DLQ > 0 o tasa de errores 5xx > 5 % durante 1 minuto) que se dispare durante un experimento de C1.

- Evidencias: **F7-E9** tablero; **F7-E10** búsqueda por `request_id`; **F7-E11** traza; **F7-E12** alerta disparada.

6.4. C4 – Seguridad de la Plataforma y Cadena de Suministro (4 pts)

- Escaneo de todas las imágenes con **Trivy**: cero vulnerabilidades críticas, o cada una justificada en el registro de hallazgos.
- Generar el **SBOM** de cada imagen (Syft o `docker sbom`) y versionarlo.
- **Rotar el token** de API de BookStack sin reconstruir imágenes y sin pérdida de servicio, documentando el procedimiento.
- Ampliar el registro de hallazgos de la sección 7.5 del TP I con al menos dos hallazgos nuevos (corregido o aceptado como riesgo, con justificación).
- Evidencias: **F7-E13** informe Trivy; **F7-E14** rotación de token.

6.5. C5 – Matriz de Pruebas, Demostración y Defensa (5 pts)

- Extender la matriz consolidada (Tabla 5 del TP I) con las pruebas **P26 en adelante**, una por evidencia F5 a F7, con resultado verificable (ver sección 8).
- **Demostración en vivo de 15 minutos**: despliegue desde cero en un comando, editar una página y encontrarla en `/v2/search`, matar un servicio y mostrar la recuperación en Grafana.
- **Preguntas individuales**: cada integrante debe poder explicar cualquier servicio, no solo el que construyó. La nota de C5 es individual.

6.6. Rúbrica del Bloque III

Ítem	Pts	Logro completo	Logro parcial (≈ 50 %)
C1 Resiliencia y caos	8	Cinco experimentos con hipótesis, MTTR medido y resultado contrastado	Menos de cuatro experimentos o sin medición de recuperación
C2 Escalabilidad	6	Comparación monolito vs. servicio en dos niveles de carga y escalado de *workers* interpretado	Una sola medición sin comparación con la línea base
C3 Observabilidad	7	RED por servicio, logs correlacionados, traza de punta a punta y alerta disparada	Solo métricas o solo logs, sin correlación
C4 Seguridad	4	Trivy + SBOM + rotación de token + hallazgos registrados	Escaneo sin tratamiento de hallazgos
C5 Matriz y defensa	5	Matriz completa, demo sin fallos y respuestas individuales sólidas	Demo parcial o integrantes que solo conocen su parte
Total Bloque III	**30**		

Tabla 9: Rúbrica de evaluación del Bloque III – Fase 7.

7. Distribución de Puntaje, Entregables y Reglas

7.1. Distribución de los 100 Puntos

Bloque	Fase	Enfoque	Ítems	Puntos
I	Fase 5	Diseño y descomposición	A1–A5	25

Bloque	Fase	Enfoque	Ítems	Puntos
II	Fase 6	Construcción, contenerización y despliegue distribuido	B1–B5	45
III	Fase 7	Operación cloud native, resiliencia, seguridad y defensa	C1–C5	30
Total				**100**

Tabla 10: Distribución del puntaje por bloque.

Los criterios de aceptación CA-1 a CA-7 son condición necesaria: si alguno no se cumple, el puntaje total se limita a 50 puntos, independientemente de la suma.

7.2. Entregables

- **Repositorio** `tp-arqweb-equipo03`, carpeta `cna/` con esta estructura mínima: `services/<servicio>/` (código, Dockerfile, pruebas), `contracts/`, `deploy/` (Compose, stack o manifiestos), `observability/` (tableros y reglas), `docs/` (C4, secuencias, ADR) y `README.md` con instrucciones de despliegue en un comando.
- **Etiqueta Git** `v2.0.0` sobre el commit entregado; historial con commits de todos los integrantes.
- **Informe en LaTeX** con la misma plantilla del TP I, continuando la numeración: secciones nuevas para las Fases 5, 6 y 7, ADR-006 a 010, evidencias F5-E1 a F7-E14 y matriz P26 en adelante.
- **Video de respaldo** (máximo 10 minutos) con la demostración, por si la demo en vivo falla por causas de infraestructura.

7.3. Hitos de Entrega

1. **Semana 2 – Revisión del Bloque I:** contratos, ADR y diagramas. Se devuelve con observaciones; el puntaje de A1–A5 puede mejorarse hasta la entrega final.
2. **Semana 5 – Entrega del Bloque II:** despliegue funcionando y verificable por el docente desde la red del laboratorio.
3. **Semana 6 – Entrega final y defensa:** informe, etiqueta `v2.0.0`, demostración y preguntas individuales.

7.4. Reglas

- Se mantienen los equipos del TP Integrador I.
- Prohibido modificar el código fuente de BookStack: toda integración va por API REST y Webhooks.
- Prohibido exponer al exterior cualquier puerto que no sea 22 o 443 de VM-1 (80 solo para la redirección 301).
- Cada evidencia debe mostrar el *prompt* de la terminal con usuario y hostname, igual que en el TP I, para identificar en qué nodo se ejecutó.
- Cualquier servicio o herramienta alternativa a las sugeridas se acepta si se justifica en un ADR.

8. Matriz de Evidencias Esperadas

La siguiente matriz continúa la Tabla 5 del TP I. Cada evidencia se convierte en una prueba de la matriz consolidada del informe, numerada desde P26, con su figura y resultado.

Prueba	Evidencia	Fase	Ítem	Descripción
P26	F5-E1	Fase 5	A1	Tablas por módulo vía `information_schema`
P27	F5-E2	Fase 5	A1	Matriz de acoplamiento módulo × tabla
P28	F5-E3	Fase 5	A1	Mapa de contextos (DDD)
P29	F5-E4	Fase 5	A3	Contratos OpenAPI/AsyncAPI validados sin errores
P30	F5-E5	Fase 5	A4	Diagrama C4 nivel 1 (contexto)
P31	F5-E6	Fase 5	A4	Diagrama C4 nivel 2 (contenedores)
P32	F5-E7	Fase 5	A4	Diagramas de secuencia con caminos de error
P33	F6-E1	Fase 6	B1	Nodos activos del clúster o motor Docker
P34	F6-E2	Fase 6	B1	`ufw status verbose` de cada nodo
P35	F6-E3	Fase 6	B1	Certificado X.509 con doble SAN
P36	F6-E4	Fase 6	B2.1	Búsqueda vía gateway antes/después de editar
P37	F6-E5	Fase 6	B2.2	Idempotencia ante evento duplicado
P38	F6-E6	Fase 6	B2.3	Ciclo completo de un job de exportación
P39	F6-E7	Fase 6	B2.3	Objeto exportado en MinIO
P40	F6-E8	Fase 6	B2.4	Subida y miniatura vía gateway
P41	F6-E9	Fase 6	B3	Etiquetas y tamaños de imágenes
P42	F6-E10	Fase 6	B3	`docker inspect`: usuario, límites, healthcheck
P43	F6-E11	Fase 6	B4	Webhook configurado en BookStack
P44	F6-E12	Fase 6	B4	Evento recibido en el broker
P45	F6-E13	Fase 6	B4	Mensaje enviado a la DLQ
P46	F6-E14	Fase 6	B5	`nginx -t` y *server block* del gateway
P47	F6-E15	Fase 6	B5	Respuesta 429 por rate limiting
P48	F6-E16	Fase 6	B5	Despliegue en un comando, servicios `healthy`
P49	F7-E1	Fase 7	C1	X1: caída de S1 y MTTR
P50	F7-E2	Fase 7	C1	X2: cero pérdida de eventos en S2
P51	F7-E3	Fase 7	C1	X3: reintento de job tras caída de *worker*
P52	F7-E4	Fase 7	C1	X4: apagado de VM-2 y convergencia
P53	F7-E5	Fase 7	C1	X5: timeout, *backoff* e interruptor de circuito
P54	F7-E6	Fase 7	C2	Resultados k6: monolito vs. S1
P55	F7-E7	Fase 7	C2	Escalado de `export-worker` 1 → 3
P56	F7-E8	Fase 7	C2	Consumo de recursos por nodo
P57	F7-E9	Fase 7	C3	Tablero RED en Grafana
P58	F7-E10	Fase 7	C3	Logs correlacionados por `request_id`
P59	F7-E11	Fase 7	C3	Traza distribuida en Jaeger
P60	F7-E12	Fase 7	C3	Alerta disparada
P61	F7-E13	Fase 7	C4	Informe Trivy y SBOM
P62	F7-E14	Fase 7	C4	Rotación del token de API

Tabla 11: Matriz de evidencias esperadas del TP Integrador II – Fases 5 a 7.